Sampling in Score-Based Generative Models: Euler–Maruyama vs Predictor–Corrector on MNIST

Hazem Ajlan

November 25, 2025

Abstract

Score-based generative models generate data by first corrupting samples into noise, then learning how to reverse this process. In this project, I implement a variance-exploding (VE) score-based model on the MNIST dataset; then, I compare Euler–Maruyama sampling with Predictor–Corrector sampling under matched numbers of score-function evaluations. The score network is trained using continuous-time denoising score matching, and all samplers use the same trained model and final Tweedie denoising step. The experiments show that, in this MNIST setting, Langevin corrector steps do not improve Fréchet Inception Distance (FID) at matched compute. Instead, spending the same score-evaluation budget on additional Euler–Maruyama predictor steps gives better sample quality. This suggests that the value of Predictor–Corrector sampling depends on the model, dataset, SDE, and compute regime.

Contents

1 Introduction **2**

2 Mathematical Background **2**

2.1 Forward SDE . . . 2
2.2 Reverse-Time SDE . . . 3
2.3 Denoising Score Matching . . . 4
2.4 Gaussian Perturbation for the VE SDE . . . 4
2.5 Sampling and Matched Compute . . . 5

3 Experimental Setup and Implementation **5**

3.1 Dataset . . . 5
3.2 Score Network Architecture . . . 5
3.3 Training Objective . . . 6
3.4 Euler–Maruyama Sampler . . . 7
3.5 Predictor–Corrector Sampler . . . 7
3.6 Final Tweedie Denoising . . . 8
3.7 Matched-NFE Comparison . . . 8
3.8 SNR Tuning . . . 8
3.9 FID Evaluation . . . 8
3.10 Sanity Check . . . 9

4 Results and Discussion **9**

4.1 Training Behavior . . . 9
4.2 Signal-to-Noise Ratio Tuning for Predictor–Corrector Sampling . . . 9
4.3 Matched-NFE Sampler Comparison . . . 10
4.4 Sanity Check: PC with $M = 0$ . . . 12
4.5 Interpretation . . . 12

5 Limitations **13**

1 Introduction

Score-based generative models work like this: define a forward process that gradually adds noise to data until it becomes a simple distribution (e.g., Gaussian noise), and then learn to reverse this process to generate new data. *Stochastic Differential Equations* (SDEs) provide the continuous-time framework that generalizes the noise-adding and noise-reversing processes. In this project, I study this idea in the setting of a "variance-exploding" SDE model trained on the MNIST dataset, with a focus on the numerical sampling stage.
Section 2 introduces the necessary math: forward SDEs, reverse-time SDEs, denoising score matching, and the matched-compute sampling setup. Section 3 describes the implementation, including the score network, training objective, Euler–Maruyama sampler, Predictor–Corrector sampler, SNR tuning, and FID evaluation. Section 4 presents the experimental results and interprets the matched-NFE comparison. Section 5 discusses the main limitations of the project.

Contributions. The main contributions of this project are:

- A from-scratch PyTorch implementation of a variance-exploding score-based generative model trained on MNIST. Euler–Maruyama matches the official PyTorch code. I build the Predictor–Corrector samplers to match the official paper algorithm.
- A matched-compute (NFE) comparison between the 2 samplers across four compute budgets and 4 random seeds, with separate SNR tuning for each Predictor–Corrector configuration.
- A sanity check verifying that the Predictor–Corrector implementation reduces to Euler–Maruyama in the no-corrector case.
- An observation that, in this setting, Predictor–Corrector sampling underperforms Euler–Maruyama at every compute budget tested.

2 Mathematical Background

2.1 Forward SDE

Let $(W_t)_{t\in[0,T]}$ be a d-dimensional Brownian motion, and let $x(t) \in \mathbb{R}^d$ denote the state of the data at time t. For images, d is the number of pixels, so an MNIST image is a point in $\mathbb{R}^{784}$.
A *diffusion process* is a continuous-time stochastic process whose state changes through many small random fluctuations. *Stochastic differential equations* (SDEs) provide a convenient way to define such processes. In this setting, we describe the evolution of $x(t)$ by

$$dx = \underbrace{f(x,t)\,dt}_{\text{deterministic drift}} + \underbrace{g(t)\,dW_t}_{\text{random noise}} . \tag{1}$$

Saying that $x(t)$ "solves" this SDE means that $x(t)$ is a random path whose changes are governed by the two terms above. The drift term describes how the state would move without randomness, as in an ordinary differential equation. The second term injects Brownian noise, scaled by the diffusion coefficient $g(t)$.
A useful way to understand the SDE is to discretize time. If Δt is small, then the Brownian increment satisfies

$$W_{t+\Delta t} - W_t \sim \mathcal{N}(0, \Delta t\, I),$$

where I is the $d \times d$ identity matrix. Equivalently,

$$W_{t+\Delta t} - W_t \approx \sqrt{\Delta t}\, z, \qquad z \sim \mathcal{N}(0, I).$$

Thus, the SDE behaves approximately like the update

$$x(t + \Delta t) \approx x(t) + f(x(t), t)\Delta t + g(t)\sqrt{\Delta t}\, z, \qquad z \sim \mathcal{N}(0, I).$$

In this sense, an SDE is the continuous-time limit of repeatedly applying small noisy updates.
In score-based generative modeling, the forward SDE is chosen to gradually add noise to the data distribution. It transforms

$$x(0) \sim p_0 \qquad \longrightarrow \qquad x(T) \sim p_T,$$

where p_0 is the data distribution and p_T is a tractable noise distribution. For MNIST, $x(0)$ is a digit image, while $x(T)$ is approximately Gaussian noise.
In this project, I use the variance-exploding (VE) SDE

$$dx = \sigma^t \, dW_t. \tag{2}$$

The process has no deterministic drift; it only adds Brownian noise. Since σ^t increases exponentially with time, the distribution $p_t(x)$ becomes increasingly diffuse, eventually "exploding" the variance of the data toward a high-noise regime.

2.2 Reverse-Time SDE

In generation, conversely, we start from noise and move back toward data. According to Anderson [2], the time reversal of a diffusion process is also a diffusion process. For the forward SDE

$$dx = f(x, t)\, dt + g(t)\, dW_t,$$

the reverse-time SDE is

$$dx = \left[f(x, t) - g(t)^2 \nabla_x \log p_t(x)\right] dt + g(t)\, d\bar{W}_t, \tag{3}$$

where $\bar{W}_t$ denotes Brownian motion when time runs backward. The gradient of log probability density

$$\nabla_x \log p_t(x)$$

is called the *score*. It points towards regions of higher probability density under the noisy distribution p_t. Therefore, if we knew the score for every t, we could sample

$$x(T) \sim p_T$$

and solve the reverse SDE backward to obtain a sample from the data distribution p_0.
For the VE SDE, since $f = 0$ and $g(t) = \sigma^t$, the reverse SDE becomes

$$dx = -\sigma^{2t} \nabla_x \log p_t(x)\, dt + \sigma^t \, d\bar{W}_t. \tag{4}$$

The true score is unknown, so we approximate it with a neural network

$$s_\theta(x, t) \approx \nabla_x \log p_t(x).$$

2.3 Denoising Score Matching

The challenge in estimating the time-dependent score is that $p_t(x)$ is unknown, so we cannot directly compute $\nabla_x \log p_t(x)$. The key trick is to use the known forward noising process.
The continuous-time denoising score matching objective is

$$\theta^* = \arg\min_\theta \mathbb{E}_{t,x(0),x(t)} \left[\lambda(t) \left\| s_\theta(x(t),t) - \nabla_{x(t)} \log p_{0t}(x(t) \mid x(0)) \right\|^2 \right]. \tag{5}$$

Note $p_{0t}(x(t) \mid x(0))$ is the distribution of the noisy sample at time t given the original sample at time 0. This objective may look counter-intuitive. If we want the score of the marginal distribution $p_t(x)$, why do we train using the score of the conditional perturbation distribution $p_{0t}(x(t) \mid x(0))$? Denoising score matching [3] tells us that, under sufficient model capacity, this still recovers the desired marginal score:

$$s_{\theta^*}(x,t) = \nabla_x \log p_t(x). \tag{6}$$

So, the training procedure is: take a clean image, add a known amount of Gaussian noise, and train the network to point back in the denoising direction.

2.4 Gaussian Perturbation for the VE SDE

For the VE SDE I use in this project, the perturbation distribution has a closed form:

$$x(t) \mid x(0) \sim \mathcal{N}\left(x(0), \frac{\sigma^{2t}-1}{2\log\sigma} I \right). \tag{7}$$

Equivalently, we can write

$$x(t) = x(0) + \sigma_t z, \qquad z \sim \mathcal{N}(0, I), \tag{8}$$

where

$$\sigma_t = \sqrt{\frac{\sigma^{2t}-1}{2\log\sigma}}. \tag{9}$$

Since the conditional distribution is Gaussian, its score can be computed explicitly. We have

$$\nabla_{x(t)} \log p_{0t}(x(t) \mid x(0)) = -\frac{x(t)-x(0)}{\sigma_t^2}. \tag{10}$$

But

$$x(t) - x(0) = \sigma_t z,$$

so this simplifies to

$$\nabla_{x(t)} \log p_{0t}(x(t) \mid x(0)) = -\frac{z}{\sigma_t}. \tag{11}$$

Thus, the target score is known during training. The network should satisfy

$$s_\theta(x(t),t) \approx -\frac{z}{\sigma_t}.$$

Equivalently,

$$s_\theta(x(t),t)\sigma_t + z \approx 0.$$

This gives the implemented loss:

$$\mathcal{L}(\theta) = \mathbb{E}\left[\left\| s_\theta(x(t),t)\sigma_t + z \right\|^2 \right]. \tag{12}$$

2.5 Sampling and Matched Compute

After training, the learned score model is used to simulate the reverse SDE. The simplest numerical method is Euler–Maruyama. For the VE reverse SDE, one backward step is

$$x_{t-\Delta t} = x_t + g(t)^2 s_\theta(x_t, t)\Delta t + g(t)\sqrt{\Delta t}\, z, \qquad z \sim \mathcal{N}(0, I). \tag{13}$$

The first term keeps the current sample, the second term moves it according to the learned score, and the third term adds the stochastic noise required by the reverse SDE.
Predictor–Corrector sampling adds another idea. At each noise level, before or after taking a reverse-SDE step, we run so-called Langevin correction steps:

$$x \leftarrow x + \eta\, s_\theta(x, t) + \sqrt{2\eta}\, z, \qquad z \sim \mathcal{N}(0, I). \tag{14}$$

The step size η is set adaptively from a signal-to-noise ratio rule that is specified next in Section 3. Unlike the predictor, these correction steps do not move to a new time level; they refine the sample at the current noise level of t.
However, corrector steps are not free. Each Euler–Maruyama predictor step and each Langevin corrector step requires one evaluation of the neural score network. Therefore, comparing samplers using the same number of time steps would be unfair. Instead, I compare them using the same number of score-function evaluations (NFE).
For Euler–Maruyama with N time steps,

$$\text{NFE} = N + 1, \tag{15}$$

where the extra 1 comes from the final Tweedie denoising step. For Predictor–Corrector sampling with P predictor steps and M corrector steps per predictor,

$$\text{NFE} = P(1 + M) + 1. \tag{16}$$

This matched-NFE comparison is the central experimental design of the project. It asks whether corrector steps improve sampling enough to justify their additional score-network evaluations.

3 Experimental Setup and Implementation

3.1 Dataset

I experiment on the MNIST dataset because it is simple enough to train quickly, but still nontrivial enough to test whether a score-based sampler can generate recognizable images. The training set is used to train the score network, and the test set is used as the real reference distribution for FID evaluation. Images are loaded as grayscale tensors in $[0, 1]$ with shape $1 \times 28 \times 28$.

3.2 Score Network Architecture

The score model $s_\theta(x, t)$ is a time-conditioned U-Net (type of Convolutional Neural Networks) with approximately 1.6M parameters. The encoder has two downsampling stages with channel widths $32 \to 64 \to 128$, followed by a bottleneck residual block and a symmetric decoder with skip connections. Each block uses group normalization with 8 groups, a SiLU nonlinearity, and 3×3 convolutions. The input is a noisy image $x(t)$ and a time t, and the output has the same shape as the image. This is necessary because the score

$$\nabla_x \log p_t(x)$$

is a gradient with respect to the image itself: for each pixel, the network outputs a direction telling the sampler how that pixel should move.
The time t is embedded using Gaussian Fourier features (using sin and cos functions) and injected into the residual blocks as a channel-wise bias. This lets the network adapt to the current noise level. At small t, the input already resembles a digit, while at large t, it may be close to pure noise. Thus, for each noise level t, the network defines a vector field

$$x \mapsto s_\theta(x, t)$$

over the image space. If you vary t, you get a time-dependent family of such vector fields. Finally, the output is divided by σ_t, since the VE conditional score has scale

$$-\frac{z}{\sigma_t}.$$

This helps the network produce outputs at the correct noise-dependent scale.

3.3 Training Objective

The model is trained using continuous-time denoising score matching. For each mini-batch, a time

$$t \sim \text{Uniform}(\varepsilon, 1)$$

is sampled independently for each image. This exposes the model to many noise levels, while the cutoff ε avoids the numerical instability of training exactly at $t = 0$.
A Gaussian noise tensor

$$z \sim \mathcal{N}(0, I)$$

is sampled, and then perturb the clean image by the VE transition kernel:

$$x(t) = x(0) + \sigma_t z, \tag{17}$$

where

$$\sigma_t = \sqrt{\frac{\sigma^{2t} - 1}{2 \log \sigma}}. \tag{18}$$

The conditional score of this perturbation kernel is

$$\nabla_{x(t)} \log p_{0t}(x(t) \mid x(0)) = -\frac{z}{\sigma_t}. \tag{19}$$

Therefore, the implemented loss is

$$\mathcal{L}(\theta) = \mathbb{E}\left[\|s_\theta(x(t), t)\sigma_t + z\|^2\right]. \tag{20}$$

Its purpose is to learn the score field needed for reverse-time generation. The model is trained for 15 epochs with Adam, learning rate 10^{-4}, and mini-batch size 32.

3.4 Euler–Maruyama Sampler

The baseline sampler is Euler–Maruyama applied to the reverse-time VE SDE. Sampling starts from the terminal prior

$$x(1) \sim \mathcal{N}(0, \sigma_1^2 I), \tag{21}$$

which corresponds to starting from noise.
The interval $[\varepsilon, 1]$ is discretized into N time steps. At each step, the sampler applies

$$x_{t-\Delta t} = x_t + g(t)^2 s_\theta(x_t, t)\Delta t + g(t)\sqrt{\Delta t}\, z, \qquad z \sim \mathcal{N}(0, I), \tag{22}$$

where $g(t) = \sigma^t$. Here, $\Delta t > 0$ is the magnitude of a backward step in time. The sign differs from the continuous reverse SDE in Eq. 4 because the sampler moves from t to $t - \Delta t$. The score term moves the sample toward higher-density regions of the noisy data distribution; the Gaussian term provides the stochasticity of the reverse diffusion. On the final step, the noise term is omitted and the deterministic mean is returned to avoid adding random noise immediately before output.

3.5 Predictor–Corrector Sampler

The Predictor–Corrector sampler uses the score model in two ways. The predictor moves the sample through time using the reverse SDE, while the corrector refines the sample at a fixed noise level. At each time level, the sampler first applies M Langevin corrector steps:

$$x \leftarrow x + \eta s_\theta(x, t) + \sqrt{2\eta}\, z, \qquad z \sim \mathcal{N}(0, I). \tag{23}$$

If $s_\theta(x, t) \approx \nabla_x \log p_t(x)$, then Langevin dynamics moves samples toward p_t. Thus, the corrector attempts to repair sampling error before the predictor moves to the next time level. The step size η is chosen so that the ratio between the deterministic score move and the random noise move is approximately equal to a "signal-to-noise" ratio (SNR). This parameter controls the Langevin corrector step size. A small SNR makes the corrector weak, while a large SNR makes the corrector more aggressive. For the Langevin update

$$x \leftarrow x + \eta s_\theta(x, t) + \sqrt{2\eta}\, z,$$

the deterministic displacement has norm approximately

$$\eta \|s_\theta(x, t)\|,$$

while the stochastic displacement has norm approximately

$$\sqrt{2\eta}\|z\|.$$

Setting their ratio equal to Signal-to-Noise Ratio (SNR),

$$\frac{\eta \|s_\theta(x, t)\|}{\sqrt{2\eta}\|z\|} = \text{SNR},$$

and solving for η gives

$$\eta = 2\left(\frac{\text{SNR} \cdot \|z\|}{\|s_\theta(x, t)\|}\right)^2.$$

So, the SNR parameter controls how strongly the corrector follows the score relative to how much random noise it injects. After the corrector steps, one Euler–Maruyama predictor step moves the sample to the next lower time level. I evaluate two configurations:

$$M = 1 \qquad \text{and} \qquad M = 2.$$

This tests whether spending more evaluations on correction improves sample quality, or whether the same compute is better spent on more predictor steps.

3.6 Final Tweedie Denoising

Both samplers use the same final Tweedie denoising step. At $t = \varepsilon$, I compute

$$\hat{x}_0 = x_\varepsilon + \sigma_\varepsilon^2 s_\theta(x_\varepsilon, \varepsilon). \tag{24}$$

This estimates the clean sample from the slightly noisy final sample. Since the same step is applied to both Euler–Maruyama and Predictor–Corrector, it does not favor either sampler.

3.7 Matched-NFE Comparison

The main comparison uses matched numbers of score-function evaluations, abbreviated NFE. NFE counts the number of forward passes through s_θ used to generate one sample. This is the natural compute unit because score-network evaluations dominate sampling runtime.
For Euler–Maruyama

$$\text{NFE} = N + 1,$$

and for Predictor–Corrector

$$\text{NFE} = P(1 + M) + 1$$

where N is the number of Euler–Maruyama steps, P is the number of predictor steps, M is the number of corrector steps per predictor, and the final $+1$ accounts for Tweedie denoising.
I use NFE budgets

$$121,\ 241,\ 481,\ 961.$$

These are chosen so that $\text{NFE} - 1$ is divisible by both 2 and 3, allowing matched-compute comparisons among Euler–Maruyama, PC with $M = 1$, and PC with $M = 2$. For example, at NFE 481, Euler–Maruyama uses 480 predictor steps; PC with $M = 1$ uses 240 predictor and 240 corrector steps; and PC with $M = 2$ uses 160 predictor and 320 corrector steps. All three use the same total score evaluations, but allocate them differently.

3.8 SNR Tuning

Before the final matched-NFE comparison, I tune the SNR separately for $M = 1$ and $M = 2$. This avoids forcing one corrector strength to serve two different numbers of corrector steps.
For each M, I sweep

$$\{0.02,\ 0.05,\ 0.08,\ 0.12,\ 0.16,\ 0.20,\ 0.25,\ 0.30\}$$

and select the value with the lowest FID on a smaller tuning run. The selected SNR is then used in the final matched-NFE experiment.

3.9 FID Evaluation

Now how can we evaluate sample quality? We use Fréchet Inception Distance (FID). It compares the mean and covariance of real and generated samples in InceptionV3 feature space. lower FID indicates a closer match in that feature space.
Since MNIST images are grayscale and low-resolution, each image is replicated to three channels and resized to 299×299 before being passed through InceptionV3. It is worth noting that FID is imperfect for MNIST because InceptionV3 is trained on natural images, not handwritten digits. Therefore, I use FID only as a relative metric between samplers under the same model, dataset, and preprocessing pipeline.

3.10 Sanity Check

As an implementation check, I verify that Predictor–Corrector reduces to Euler–Maruyama when $M = 0$. With no Langevin correction, the PC sampler consists only of Euler–Maruyama predictor steps followed by the same Tweedie denoising step. So, Euler–Maruyama with N steps and PC with $P = N$, $M = 0$ should give nearly identical FID values.

4 Results and Discussion

4.1 Training Behavior

Figure 1 shows the training loss over 15 epochs. The loss decreases steadily, indicating that the score network is learning the denoising score matching objective. The downward trend suggests that the model is learning a useful approximation to the time-dependent score function.

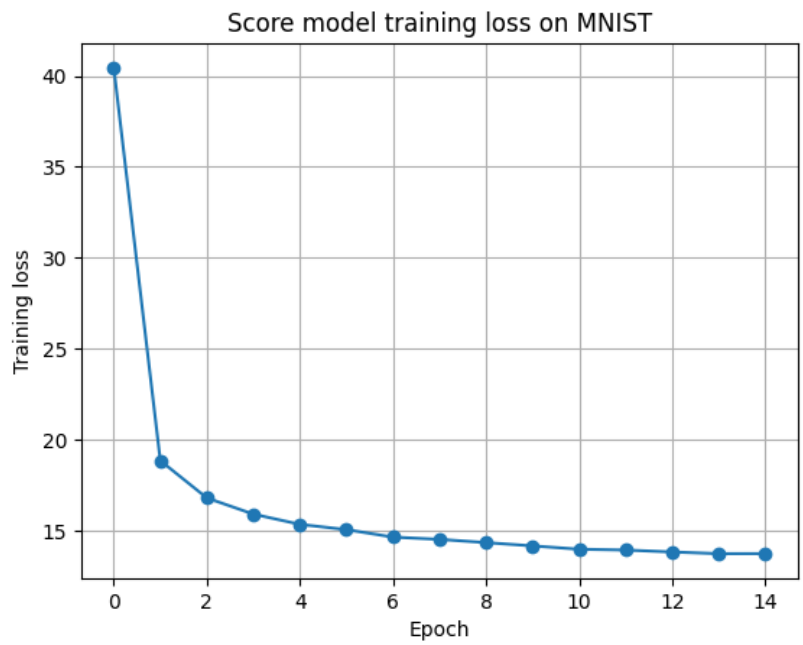

Figure 1: Training loss for the VE score model on MNIST.

The purpose of this training stage is not to minimize a reconstruction loss directly, but to learn the score field

$$s_\theta(x, t) \approx \nabla_x \log p_t(x).$$

The later sampling experiments therefore depend on both the quality of this learned score and the numerical method used to simulate the reverse-time process.

4.2 Signal-to-Noise Ratio Tuning for Predictor–Corrector Sampling

Before the final sampler comparison, I tuned the SNR for the Predictor–Corrector sampler. Figures 2 and 3 show the SNR sweeps for $M = 1$ and $M = 2$, respectively. For $M = 1$, the best value in the grid was

$$\text{SNR} = 0.25,$$

which lies in the interior of the searched range. For $M = 2$, the best value in the tested grid was

$$\text{SNR} = 0.30,$$

but here the FID is monotonically decreasing across the entire grid, and the best value lies at the right boundary. This means the true optimum likely lies outside the searched range, somewhere larger than 0.30. The two values were tuned separately because the optimal corrector strength can depend on the number of Langevin steps per predictor step.

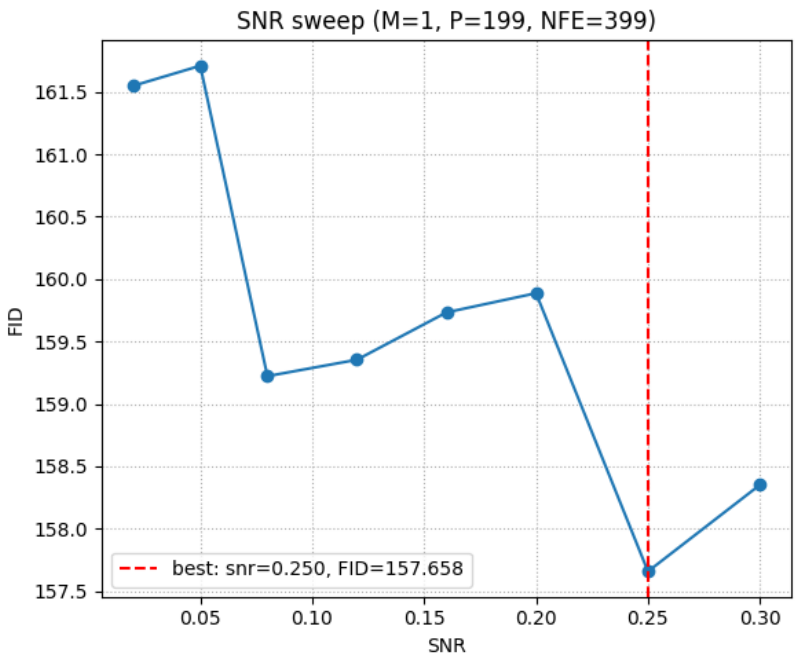

Figure 2: SNR sweep for Predictor–Corrector sampling with one corrector step per predictor step.

More importantly, observe the absolute scale of these FID values. The $M = 1$ sweep produces values between 158 and 162, while the $M = 2$ sweep produces values between 213 and 217. These are substantially worse than what Euler–Maruyama achieves at a similar NFE budget (see Table 1). The sweep does not just select a hyperparameter; it confirms that PC underperforms across the reasonable range of SNR values. A larger SNR might improve $M = 2$ further, but the gap to Euler–Maruyama is large enough that this is unlikely to change the main conclusion.

4.3 Matched-NFE Sampler Comparison

The main experiment compares Euler–Maruyama sampling against Predictor–Corrector sampling at matched numbers of score-function evaluations. The comparison is shown in Figure 4, and the numerical values are reported in Table 1.

The main observation is that Euler–Maruyama outperforms Predictor–Corrector sampling at every matched compute budget. This is the opposite of the trend reported on CIFAR-10 by Song et al. [1], where PC sampling gave lower FID than Euler–Maruyama at matched compute.

The gap is especially visible at intermediate budgets. At NFE 481, Euler–Maruyama achieves FID 11.71, while PC with $M = 1$ obtains FID 138.85 and PC with $M = 2$ obtains FID 182.23. At the largest budget (NFE 961), PC with $M = 1$ closes most of the gap but still remains noticeably worse than EM (17.76 versus 10.86).

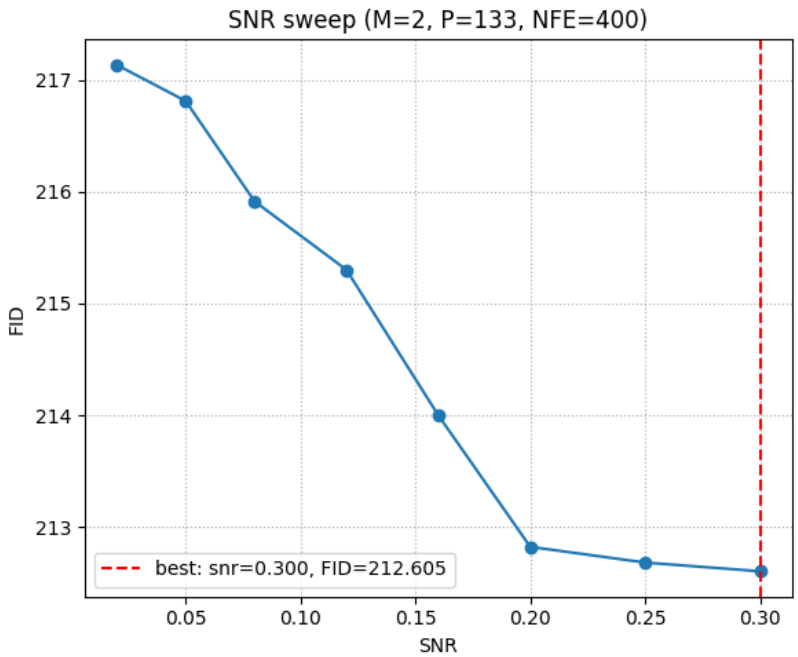

Figure 3: SNR sweep for Predictor–Corrector sampling with two corrector steps per predictor step.

NFE	Euler–Maruyama	PC ($M = 1$)	PC ($M = 2$)
121	236.16 ± 0.59	357.58 ± 0.72	378.47 ± 0.08
241	141.70 ± 0.48	232.40 ± 0.37	311.98 ± 0.50
481	11.71 ± 0.11	138.85 ± 0.32	182.23 ± 0.17
961	10.86 ± 0.24	17.76 ± 0.09	99.37 ± 0.30

Table 1: FID scores for Euler–Maruyama and Predictor–Corrector samplers at matched NFE. Values are reported as mean $\pm$ standard deviation across three seeds.

A separate but informative observation is that Euler–Maruyama itself saturates around NFE 481: improving the budget from 481 to 961 only changes its FID from 11.71 to 10.86. This suggests that, for this trained model, EM has already extracted most of the benefit of additional discretization by roughly 500 score evaluations, and the remaining error is dominated by the score model itself rather than by numerical integration.

This result does not mean that Predictor–Corrector sampling is incorrect. Rather, it shows that in this particular MNIST VE-SDE setting, the corrector steps are not the best use of a fixed score-evaluation budget. Each Langevin corrector step requires an additional score-network evaluation. Therefore, under matched NFE, using corrector steps forces the sampler to use fewer predictor steps. For example, at NFE 481, Euler–Maruyama uses 480 reverse-SDE steps, while PC with $M = 1$ uses only 240 predictor steps and 240 corrector steps. In this experiment, the additional predictor steps are more valuable than the Langevin corrections.

The result is especially clear for $M = 2$. Since this configuration spends two score evaluations on correction for every one predictor step, it has even fewer reverse-SDE predictor steps at the same total NFE. This appears to hurt performance substantially. The corrector steps may refine samples locally at a fixed noise level, but they do not replace the need to accurately move through the reverse-time trajectory.

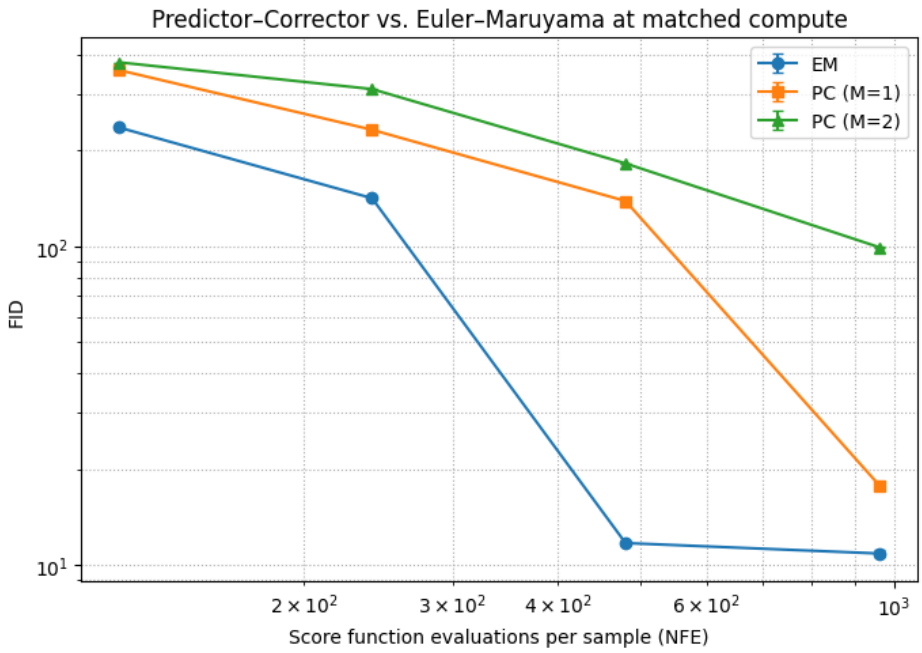

Figure 4: FID versus number of score-function evaluations per sample. Lower FID is better.

4.4 Sanity Check: PC with $M = 0$

By letting $M = 0$, the Predictor–Corrector sampler has no Langevin correction steps and should reduce to Euler–Maruyama with the same number of predictor steps, followed by the same Tweedie denoising step. At $N = P = 240$ on a single fixed seed, Euler–Maruyama gave

$$\text{FID} = 147.376,$$

while PC with $M = 0$ gave

$$\text{FID} = 147.745.$$

The 0.369 difference is negligible relative to the FID scale in this experiment. (These single-seed values are slightly higher than the three-seed averages reported in Table 1 at NFE 241, because the main experiment averages across three different seeds while this sanity check uses a single fixed seed.) This confirms that the predictor path and NFE accounting are implemented consistently. Therefore, the worse performance of PC for $M = 1$ and $M = 2$ is caused by the effect of the corrector steps under the matched-compute constraint.

4.5 Interpretation

The experiment suggests that Predictor–Corrector sampling is not automatically better than Euler–Maruyama. Its advantage depends on whether the corrector steps improve the sample distribution enough to justify their additional score evaluations. In this MNIST experiment, they do not. One explanation is that the reverse-time trajectory itself needs sufficiently fine discretization. When the number of predictor steps is reduced too much, the sampler does not move through the noise schedule accurately enough. The Langevin corrector can adjust samples at a fixed noise level, but it cannot fully compensate for a coarse reverse-time discretization.

Another explanation that I conjecture but cannot verify concerns the corrector's target distribution. By construction, Langevin dynamics with score $s_\theta(x, t)$ has a stationary distribution that depends

on s_θ, not on the true score $\nabla_x \log p_t(x)$. If the learned score differs systematically from the true score, then running the corrector closer to its stationary distribution (by using more corrector steps, or running them longer) pulls samples toward the wrong distribution rather than the right one. This is a plausible mechanism by which a larger M would hurt rather than help on an imperfect score model, and it is consistent with the $M = 2$ configuration being substantially worse than $M = 1$ in our results. However, verifying this mechanism directly requires analyzing the learned score's bias, which is beyond the scope of this project.

The main takeaway from this experiment is:

For this trained VE-SDE MNIST model, at matched NFE, extra Euler–Maruyama predictor steps are more useful than Langevin corrector steps. The reported advantage of Predictor–Corrector sampling on natural-image datasets does not automatically transfer to smaller, less-trained models on simpler data.

This is a negative result, but it is still informative. It shows that PC sampling should be evaluated under fair compute budgets rather than by simply adding corrector steps on top of a fixed number of predictor steps. Under such a matched-compute comparison, the corrector is not free, and in this setting its cost outweighs its benefit.

5 Limitations

This project has several limitations. First, all experiments are conducted on MNIST. This makes the project computationally manageable and allows the sampling behavior to be studied clearly, but MNIST is much simpler than natural image datasets such as CIFAR-10 or CelebA. The conclusion that Euler–Maruyama outperforms Predictor–Corrector sampling at matched NFE in this setting should not be interpreted as a universal statement about all score-based models.

Second, the score network is relatively small (approximately 1.6M parameters) and trained for only 15 epochs. The performance of Predictor–Corrector sampling may depend strongly on the quality of the learned score, i.e., a better-trained or larger model could change the relative usefulness of Langevin corrector steps, especially if the score-bias interpretation in Section 4.5 is correct.

Third, the comparison uses only the variance-exploding SDE. Other SDE choices, such as variance-preserving or sub-variance-preserving SDEs, may interact differently with Predictor–Corrector sampling.

Fourth, FID is an imperfect metric for MNIST. InceptionV3 is trained on natural images, not handwritten digits, so the absolute FID values should not be compared directly with standard CIFAR-10 or ImageNet results. In this project, FID is used only as a relative metric across samplers under the same preprocessing and evaluation pipeline.

Finally, the experiment compares samplers primarily by score-function evaluations. NFE is a useful proxy for compute because neural network evaluations dominate sampling cost, but it is not identical to wall-clock runtime. A more complete study could also report time per sample, memory use, and visual sample quality across datasets.

References

[1] Yang Song et al., *Score-Based Generative Modeling through Stochastic Differential Equations*, ICLR 2021.

[2] B. D. O. Anderson, "Reverse-time diffusion equation models," *Stochastic Processes and their Applications*, vol. 12, no. 3, pp. 313–326, 1982.

[3] P. Vincent, "A connection between score matching and denoising autoencoders," *Neural Computation*, vol. 23, no. 7, pp. 1661–1674, 2011.

[4] Y. Song, J. Sohl-Dickstein, D. P. Kingma, A. Kumar, S. Ermon, and B. Poole, "Score-Based Generative Modeling through Stochastic Differential Equations (Official Implementation)," GitHub repository, 2021. https://github.com/yang-song/score_sde